Update on tensor representation strategy

March 4, 2026

1 Previous work

The original approach constructed a single global image by unrolling multiple geometrical strips of the mesh into one rectangular tensor. Although computationally convenient for convolutional neural networks, this representation introduced geometric inconsistencies: certain physically adjacent regions in the mesh were not adjacent in the 2D layout.

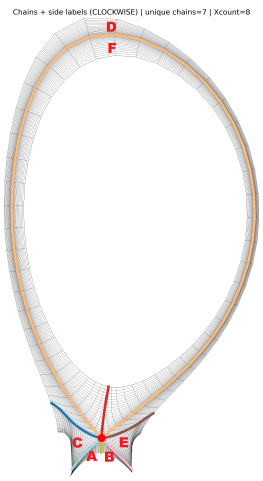
2 Limitations of the Single-Image Layout

The mesh topology contains strips that are adjacent in the physical domain but cannot all be embedded in a single planar layout without breaking at least one adjacency constraint.

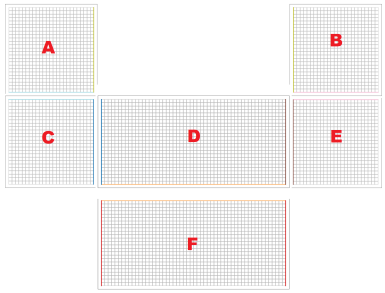
In particular:

- The bottom-center block (the nearest strip to the core, from now referred as F) is physically adjacent to the mid-center block (the strip adjacent to the external wall, from now reffered as D).
- The top-left (A) and top-right (B) blocks are also physically adjacent.

A single global canvas forces a trade-off: preserving the D & F adjacency breaks the A & B adjacency, and vice versa. This results in artificial discontinuities that may mislead convolutional filters, which assume local spatial coherence.



(a) Original mesh



(b) Previous representation, doesn't reflect real geometrical adjacency

Figure 1: representation of mesh and extended figure

3 New Three-View Tensor Strategy

To resolve this structural conflict, the tensor representation has been redesigned into three separate images, each corresponding to a geometrically consistent subset of the mesh:

- View 1: top-left (A) and top-right (B) strips.
- View 2: mid-left (C), mid-center (D), and mid-right (E) strips.
- View 3: bottom-center (F) strip.

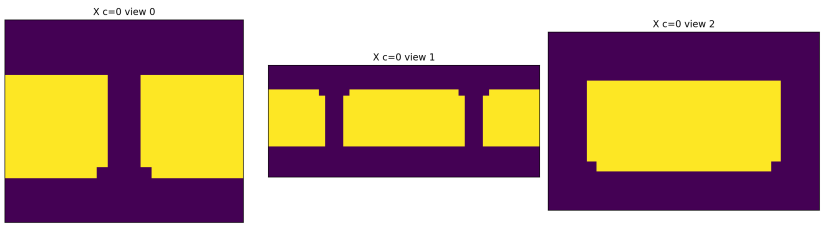


Figure 2: purple areas are the masked region, yellow is where real values are

This decomposition ensures that:

1. Physically adjacent regions remain adjacent within their respective view.
2. No artificial spatial inversions are introduced.
3. Padding and masking are applied consistently to preserve convolutional compatibility.

Each view is converted into a multi-channel tensor with:

- Broadcast global conditioning parameters,
- Optional geometry channels, (are these needed? I am adding two extra dimension that are x coordinate of centroid of the cell and y coordinate of centroid of the cell)
- Field outputs aligned with the same spatial layout,
- A binary mask to ignore padded regions during loss computation.

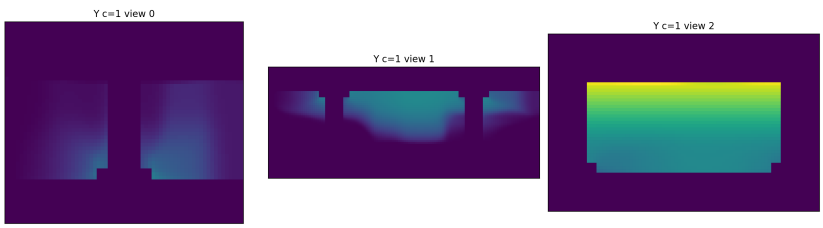


Figure 3: 3 images representation visualized with electron temperature (channel 0)

4 Future Work

The next step consists of training on the three view tensors using a U-Net architecture while explicitly enforcing cross view boundary consistency. After each convolutional operation, the border pixels corresponding to physically adjacent strips will be re-injected from their matching neighboring view. This mechanism ensures that information flow across physically connected regions is preserved, despite the presence of zero-padding used to maintain constant spatial dimensions. The padding itself will remain inactive through masking, so that artificial zero rows and columns do not contribute to the learned representation while still enabling stable convolutional operations.

5 Question

- What is the most appropriate normalization strategy for heterogeneous physical quantities in both inputs and outputs? For example, temperature values can reach magnitudes on the order of 10^{24} and are strictly positive, whereas velocities are signed and typically range between -10^5 and 10^5. Should different transformations be applied per channel to ensure numerical stability and balanced gradient contributions if yes do you have any suggestion for such transformations?

- Does it remain physically and statistically justified to include the geometrical coordinates as additional input channels? The current configuration uses 8 global conditioning parameters, 1 mask channel, and 2 coordinate channels. Should the coordinate channels be retained to provide spatial context to the convolutional filters, or could they introduce redundancy or bias given that the mesh topology is fixed?

- What UNET architecture should I use? 1. process each image indipendently with a shared Unet 2. use different Unet and merge the results at the end